

# Chapter 6

## Case Studies and Laboratory Practice

### 6.1 Case Studies

A **Case Study** is a detailed examination of a specific instance that illustrates broader operating system concepts. Case studies enable students to analyze real-world scenarios, identify underlying mechanisms, and draw conclusions about system behavior. They are particularly useful for exploring complex interactions that are difficult to isolate in controlled experiments.

### 6.2 Practical Laboratory Work

The term **Laboratory** refers to a structured **Hands-on** activity that combines guided instructions with practical tasks. This chapter includes **Practical Lab Work** that focuses on Linux-based experiments and simulation projects. Each **Exercise** is designed to reinforce understanding of a specific concept or technique.

#### 6.2.1 Lab Workflow

Setup -> Execute -> Collect -> Analyze & Report

#### 6.2.2 Hands-on Linux Exercises

The following exercise guides students through the configuration of CPU Scheduling policies on a Linux system and the collection of performance metrics.

```
# Sample script to run a round-robin benchmark
taskset -c 0 ./benchmark.sh &
for i in {1..5}; do
    taskset -c 0 ./benchmark.sh &
done
wait
```

## 6.3 Simulation of Scheduling and Synchronization Algorithms

The **Simulation** component models the behavior of scheduling and synchronization mechanisms using a computational framework. This allows exploration of scenarios that are difficult or impractical to reproduce on physical hardware.

### 6.3.1 Simulation Framework

The framework captures system state, processes events, and records outcomes. It supports both **Preemptive Scheduling** and **Non-preemptive Scheduling**, as well as **Mutex** and **Semaphore** synchronization.

```
+-----+ +-----+ +-----+ +-----+ +-----+
| New   |-->| Ready |-->| Running|-->| Waiting|-->| Terminated |
+-----+ +-----+ +-----+ +-----+ +-----+
```

Preemptive Scheduling:

```
+-----+
| CPU running P |
+-----+
      |
      v
+-----+
| Timer interrupt|
+-----+
      |
      v
+-----+
| Scheduler selects Q |
+-----+
```

Non-preemptive Scheduling:

```
+-----+
| CPU running P |
+-----+
      |
      v
+-----+
| P voluntarily yields |
+-----+
      |
      v
+-----+
```

```

| Scheduler selects Q |
+-----+

```

## Simulation Process

Simulation Process

```

|
v
+-----+
| Load Program      |
+-----+
|
v
+-----+
| Execute Instructions|
+-----+
|
v
+-----+
| Record Events      |
+-----+
|
v
+-----+
| Analyze Results    |
+-----+
|
v
+-----+
| Generate Report    |
+-----+

```

## Example: Round-Robin Scheduling Simulation

```

# Pseudo-code for a round-robin simulator
processes = [P1, P2, P3, P4]
quantum   = 4
ready_q   = queue(processes)

while not ready_q.empty():
    p = ready_q.dequeue()
    execute(p, min(quantum, p.remaining_time))
    if p.remaining_time > quantum:
        p.remaining_time -= quantum

```

```

        ready_q.enqueue(p)
    else:
        p.remaining_time = 0
        p.state = TERMINATED
        record(p)

```

[ P1 ] -> [ P2 ] -> [ P3 ] -> [ P4 ] -> (loop)

| Approach   | Key Characteristics                                                                     |
|------------|-----------------------------------------------------------------------------------------|
| Case Study | In-depth analysis of a single instance; focuses on context and detailed observation.    |
| Laboratory | Controlled setting for repeated experimentation; emphasizes hands-on execution.         |
| Simulation | Abstract model that mimics system behavior; enables exploration beyond physical limits. |

## 6.4 Conclusion

The practical experiences described in this chapter provide a solid foundation for further research in system optimization and advanced operating system design. The techniques and methodologies introduced here will be built upon in future studies involving performance modeling and system verification.

## Tutorial Questions

1. Describe the purpose of a case study in operating systems education. Provide an example from this chapter.
2. Explain the steps involved in conducting a hands-on Linux exercise to evaluate CPU Scheduling.
3. Compare two scheduling algorithms using a simulation scenario. Which algorithm performed better under high load and why?
4. Identify a common race condition that can arise during a synchronization simulation and propose a synchronization primitive to prevent it.
5. Explain how a monitor can be used to enforce mutual exclusion in a simulated critical section.
6. Discuss the role of a lab report in documenting the results of a scheduling simulation. List its essential sections.
7. Provide an example of a practical lab exercise that investigates virtual memory paging. What metrics would you measure?

8. Explain the difference between preemptive and non-preemptive scheduling in the context of a simulation. How does each affect the simulation logic?